

Parametertyper

I de exempel vi hittills behandlat har den normala metoden för parameteröverföring använts, nämligen *värdeparametrar*. Detta betyder att då en funktion anropats med en parameter har endast variabelns värde kopierats till funktionen. Den parameter som funktionen sedan använder har ingenting gemensamt med den variabel/konstant som värdet ursprungligen kom ifrån, den har helt enkelt kopierats.

```
#include <iostream>
void Test (int TestTal) {
    TestTal = 10;
    cout << "TestTal är: " << TestTal << endl;
}

int main () {
    int Tal = 5;

    cout << "Tal är: " << Tal << endl;
    Test ( Tal );
    cout << "Tal är: " << Tal << endl;
}
```

Man kunde lätt ledas att tro att funktionen `Test` ovan kan ändra värdet på parametern `TestTal` till 10, och samma värde skulle bli aktivt i `HuvudProgram`. Så är dock inte fallet, utan i `HuvudProgram` kommer `Tal` att fortfarande ha värdet 5. Vill vi att `Tal` skall ändra värde måste vi med våra kunskaper hittills returnera ett värde från `Test` och spara det i `Tal`. Körning av programmet ovan skulle ge:

```
% ParameterTest1
Tal är: 5
TestTal är: 10
Tal är: 5
%
```

Referensparametrar

Det finns ett alternativt sätt (egetligen två) att göra att `Test` kan ändra värdet på sin parameter så att det även reflekteras i den anropande funktionen. Denna funktionalitet åstadkomms genom s.k. *referensparametrar*, då man istället för en kopia av en variabel skickar en *referens* till en variabel till funktionen. För att göra en parameter till en referensparameter i en funktion räcker det med att skriva ett `&` mellan variabelns datatyp och namn i funktionens parameter lista. Vi kan nu skriva om ovanstående program så att det verkligen ändrar värdet på parametern.

```
#include <iostream>
void Test (int & TestTal) {
    TestTal = 10;
    cout << "TestTal är: " << TestTal << endl;
}

int main () {
    int Tal = 5;
```

```

cout << "Tal är: " << Tal << endl;
Test ( Tal );
cout << "Tal är: " << Tal << endl;
}

```

Enda skillnaden i det modifierade exemplet är att vi har lagt till ett `&`-tecken. Detta kan göras för vilka datatyper som helst. Vi kommer att använda referensparametrar mera senare då vi behandlar klasser. Referensparametrar kan vara väldigt praktiska, t.e.x om man har en funktion som behöver returnera flera än ett värde. Då kan man skicka med variabler till funktionen dit resultatet placeras. Körning av programmet ovan skulle ge:

```

% ParameterTest2
Tal är: 5
TestTal är: 10
Tal är: 10
%

```

Konstanta parametrar

Ibland finns det behov för att kunna garantera att en parameter som skickas till en funktion inte ändras i funktionen. Speciellt då man använder sig av referensparametrar är det viktigt att kunna försäkra sig om att en viss funktion inte ändrar på värdet. Varför skickar man då data som referensparametrar, vore det inte bättre att skicka den "helt normalt" i så fall? Orsaken (som vi skall se senare i [avsnittet Sammansatta datatyper som parametrar i Kapitel 9](#)) är effektivitet. Då man använder komplicerade datatyper är det mycket effektivare att skicka data som referensparametrar jämfört med att skicka dem som normala parametrar, eftersom mycket mindre data kopieras för funktionsanropet. I sådana fall vill man ha effektivitet, men även säkerhet. Detta kan uppnås genom att placera nyckelordet `const` framför parametern. I exemplet i förra kapitlet behövde vi ju egentligen inte manipulera parametern `TestTal` på något sätt, så då kunde vi ha gjort den till en konstant parameter istället:

```

void Test (const int & TestTal) {
    TestTal = 10;
    cout << "TestTal är: " << TestTal << endl;
}

void HuvudProgram () {
    int Tal = 5;

    cout << "Tal är: " << Tal << endl;
    Test ( Tal );
    cout << "Tal är: " << Tal << endl;
}

```

Enda skillnaden är ett extra `const`. Nu kommer kompilatorn att ge ett felmeddelande om man försöker ändra värdet på `TestTal` på något sätt. Senare då vi behandlar klasser (se [Kapitel 14](#)) kommer vi att använda `const` i mycket hög grad för att skydda parametrar.

Det kan anses vara god programmeringssed att använda `const` för parametrar som kan ändras av en funktion, men som funktionen inte har för avsikt att ändra. Det gör det lättare att läsa ett program då man snabbt kan se vilka parametrar som ändras av en funktion.

Om man har en konstant variabel (se [avsnittet Konstanter i Kapitel 2](#)) och denna skickas som referensparamameter *måste* man ha nyckelordet `const` vid parameterdefinitionen, annars ger kompilatorn ett felmeddelande. Följande program är illegalt:

```
void testaKonstant (int & Tal) {  
    // gör någonting  
}  
  
int main () {  
    const int Test = 10;  
  
    // anropa en konstant funktion  
    testaKonstant ( Test );  
}
```

Funktionen `testaKonstant()` har en referensparameter, och kan således ändra på värdet på `Tal`, men det tal som skickas är en konstant. I detta fall krävs således definitionen `const` för funktionsparametern, och `testaKonstant()` bör skrivas:

```
void testaKonstant (const int & Tal) {  
    // gör någonting  
}
```

Samma förfarande gäller all data som är konstant på ett eller annat sätt.

[Förutgående](#)

Anropa funktioner

[Hem](#)

[Upp](#)

[Nästa](#)

Funktionen `main()`